Deliverable 2

In [ ]:
```python
# Imports
import re
import json

import pandas as pd
import numpy as np

from pathlib import Path
from charset_normalizer import from_path
```

Task 1

In [ ]:
```python
def file_reader(file_path: Path) -> tuple[str, str]:
    """
    Reads a file as bytes, checking its encoding. Decodes the bytes
    into text, and returns a tuple of two strings.

    args:
        file_path: The path for the file to be checked

    returns:
        decoded_text: the decoded text as string
        encoding: the encoding of the given text

    """
    raw_bytes = Path(file_path).read_bytes()

    # Checking for utf-8 encoding with BOM
    if raw_bytes[:3] == b"\xef\xbb\xbf":
        encoding = "utf-8-sig"

    # Otherwise try to guess correct encoding
    else:
        best_match = from_path(file_path).best()

        if best_match is None:
            raise ValueError(f"Could not detect encoding of {file_path}")

        encoding = best_match.encoding

    decoded_text = raw_bytes.decode(encoding)

    return decoded_text, encoding
```

For the log_parser() I've used lecture 6 for regex syntax

In [53]:
```python
def log_parser(decoded_text: str) -> list[dict[str, str | float]]:
    """
    Uses regular expression to capture 6 key fields from valid lines in a log file
```

```
    ignores noise lines.

    Key fields:
        timestamp
        register_id
        item
        price
        currency
        status

    args:
        decoded_text: a string of a decoded file.

    returns:
        list: List of dictonaries with valid transaction record
    """

    verbose_pattern = re.compile(
        r"""
        \[(?P<timestamp> \d{4}-\d{2}-\d{2}\ \d{2}:\d{2}:\d{2})\]    # datetime YYYY
        \s+                                                         # whitespace
        \[(?P<register_id> POS_\d{2})\]                             # POS_xx
        \s+                                                         # whitespace
        item=(?P<item>\S+)                                          # item
        \s+                                                         # whitespace
        price=(?P<price>\S+)                                        # price
        \s+                                                         # whitespace
        (?P<currency>[A-Za-z]{3})                                   # currency [xxx
        \s+                                                         # whitespace
        \( (?P<status>[\w-]+) \)                                    # status
        $
    """,
        re.VERBOSE,
    )

    valid_records = []
    for line in decoded_text.splitlines():
        m = verbose_pattern.match(line)

        # check for non-valid lines
        if not m:
            continue

        valid_records.append(
            {
            "timestamp": m.group("timestamp"),
            "register_id": m.group("register_id"),
            "item": m.group("item"),
            "price": float(m.group("price")), # converts str to float here
            "currency": m.group("currency"),
            "status": m.group("status"),
            }
        )

    return valid_records
```

Task 2

In [ ]:
```python
def recursive_dir_search(input_path: Path | str) -> list[Path]:
    """
    Recursively search through a directory hierarchy finding .log files.

    arg:
        input_path: This can be a path object or a string for the location for wher
        to start the search.

    return:
        path_list: A list with the path to the found .log files.
    """
    inp_pat = Path(input_path)

    path_list = []
    for i in inp_pat.iterdir():

        if i.is_dir():
            path_list.extend(recursive_dir_search(i))

        elif i.is_file() and i.suffix == ".log":
            path_list.append(i)

    return path_list
```

In [55]:
```python
def traverse_nested(data: dict | list) -> set[str]:

    """
    Goes through a nested file of dicts and lists,
    recursivly checking for strings where "active": true.

    arg:
        data: parsed json data

    return:
        active_id: a set of register id strings where "active": true

    """
    active_id = set()

    if isinstance(data, dict):
        if data.get("active") is True and isinstance(data.get("id"), str):
            active_id.add(data["id"])

        for value in data.values():
            active_id.update(traverse_nested(value))

    elif isinstance(data, list):
        for item in data:
            active_id.update(traverse_nested(item))

    return active_id
```

Task 3

In [56]:
```python
# Task 3.1 Remove Price Outliers
def remove_price_outliers(prices: list[float] | np.ndarray) -> np.ndarray:
    """Converts input price data to a 1D Numpy float array, detects hardware glitch

    outliers (|x - mu| > 3 * sigma) and replaces them with the median of valid pric

    args:
        prices: List or NmuPy array of transaction prices values.

    returns:
        cleaned_prices: 1D NumPy float array with outliers replaces by valid median
    """

    # 1. Convert input data to a 1D NumPy float array
    price_arr = np.asanyarray(prices, dtype=float).ravel()

    if price_arr.size == 0:
        return price_arr


    # 2. Calculate mean (mu) and standard deviation (sigma)
    mean = np.mean(price_arr)
    std = np.std(price_arr)


    # 3. Detect outliers (|x - mean| > 3 * std)
    outliers_mask = np.abs(price_arr - mean) > (3 * std)


    # 4. Calculate the median of valid (non-outlier) prices
    valid_prices = price_arr[~outliers_mask]
    if len(valid_prices) > 0:
        valid_median = np.median(valid_prices)
    else:
        valid_median = np.median(price_arr)

    # 5. Replace outliers with the valid median and return cleaned array
    cleaned_prices = price_arr.copy()
    cleaned_prices[outliers_mask] = valid_median

    return cleaned_prices

# Task 3.2 Compute Refund Ratio
def compute_refund_ratio(
        sales_count: np.ndarray, refund_count: np.ndarray
    )    -> np.ndarray:
    """Computes the element wise refund ratio using Numpy array operations:

    Refund Ratio = Refund Count / (Sales Count + 10^-6)

    args:
        sales_count: Numpy array containing sales counts per item.
        refund_count: Numpy array containing refund counts per item.
```

```
    returns:
        refund_ratio: Numpy array of calculated refund ratios.
    """

    sales = np.asarray(sales_count, dtype=float)
    refunds = np.asarray(refund_count, dtype=float)

    epsilon = 1e-6

    return refunds / (sales + epsilon)
```

Task 4

In [57]:
```
def create_dataframe(transactions: list[dict]) -> pd.DataFrame:
    """
    Creates a Pandas DataFrame from a list of transaction dictionaries.

    args:
        transactions: List of dictionaries containing transaction data.

    returns:
        df: DataFrame with a DatetimeIndex named timestamp, sorted chronologically.
    """

    df = pd.DataFrame(transactions)

    df["timestamp"] = pd.to_datetime(df["timestamp"], format="%Y-%m-%d %H:%M:%S", e

    df.set_index("timestamp", inplace=True)
    df.sort_index(inplace=True)

    return df
```

In [58]:
```
def hourly_revenue(df: pd.DataFrame) -> pd.DataFrame:
    """
    Calculates the hourly revenue for each item in the DataFrame.

    args:
        df: Transaction DataFrame with a DatetimeIndex and columns item and price.

    returns:
        hourly: DataFrame with an (item, timestamp) MultiIndex and a price
            column containing the summed prices per item per hour.

    """

    grouped = df.groupby("item")
    hourly = grouped[["price"]].resample("1h").sum()

    return hourly
```

In [59]:
```
def status_pivot_table(df: pd.DataFrame) -> pd.DataFrame:
    """
    Sums transaction prices per item across status categories.
```

```
    args:
        df: Transaction DataFrame containing item, status, and price columns.

    returns:
        pivot: DataFrame with items as rows, statuses as columns and summed
            prices as values.
    """

    pivot = df.pivot_table(index="item", columns="status", values="price", aggfunc=

    return pivot
```

In [60]:
```
def item_refund_ratio(df: pd.DataFrame) -> pd.DataFrame:
    """
    Calculates the refund ratio per item using compute_refund_ratio.

    args:
        df: Transaction DataFrame containing item and status columns.

    returns:
        refund_ratio_df: DataFrame indexed by item with a refund_ratio column.
    """

    counts = df.groupby(["item", "status"]).size().unstack(fill_value=0)
    sales_count = counts.get("SUCCESS", pd.Series(0, index=counts.index))
    refund_count = counts.get("REFUND", pd.Series(0, index=counts.index))

    sales_arr = sales_count.to_numpy()
    refund_arr = refund_count.to_numpy()

    refund_ratios = compute_refund_ratio(sales_arr, refund_arr)

    refund_ratio_df = pd.DataFrame({"refund_ratio": refund_ratios}, index=counts.in

    return refund_ratio_df
```

Task 5

In [61]:
```
# 1. Find all log files and extract the active register IDs.
data_path = Path("data")

log_files = recursive_dir_search(data_path)

with open(data_path / "store_config.json", encoding="utf-8") as f:
    store_config = json.load(f)

active_registers = traverse_nested(store_config)

# 2. Read log files and collect transaction records
transactions = []

for log_path in log_files:
    text, encoding = file_reader(log_path)
    records = log_parser(text)
```

```
    transactions.extend(records)

# 3. Create the initial DataFrame
df = create_dataframe(transactions)

# 4. Clean price outliers before correcting to NOK prices
df["price"] = remove_price_outliers(df["price"].to_numpy())

# 5. Correct prices recorded in NOK
nok_mask = df["currency"] == "NOK"
df.loc[nok_mask, "price"] = df.loc[nok_mask, "price"] * 10

# 6. Generate reports using the functions from task 4
hourly = hourly_revenue(df)

pivot = status_pivot_table(df)
pivot = pivot.reindex(columns=["SUCCESS", "REFUND", "CANCELLED"], fill_value=0)

ratios = item_refund_ratio(df)

# 7. Save output files
output_dir = Path("output_data")

df.to_csv(output_dir / "cleaned_transactions.csv")

hourly.reset_index().to_json(
    output_dir / "hourly_revenue.json",
    orient="records",
    date_format="iso",
    indent=2
)

pivot.to_csv(output_dir / "status_pivot_table.csv")
ratios.to_csv(output_dir / "item_refund_ratios.csv")
```